

Turning Software-Development Knowledge Into AI Context

Using proven processes and historical approvals to guide more reliable AI-assisted implementation.

01

WHAT ALREADY EXISTED BEFORE AI

Teams already had a working SDLC structure



Analyst captures client requirement



User stories + acceptance criteria define goals



Documentation / tickets record the work



Developer builds



QA validates



The structure already existed before AI; it was not a legacy-code problem.

02

PROBLEMS FACED

Why direct AI usage created gaps



AI was used without full context



Different team members interpreted requirements differently



Structure inputs were not being passed into AI



AI lacked clarity on goals, permissions, and boundaries



Historical approved work was not being reused



Direct prompting without context led to inconsistent results

03

HOW WE SOLVED IT

Connect the existing process to AI

STRUCTURED INPUTS FED TO AI



Requirements



User Stories



Acceptance
Criteria



Goals



Permissions /
Degree of access



Historical merged
PRs



QA-cleared
outputs



Customer-
approved outputs

OUTCOMES



AI-assisted development



AI-assisted QA support



Better aligned output



We reused the existing SDLC structure instead of bypassing it

04

REAL-WORLD SCENARIO

How the structured flow works in practice



Analyst gathers the
requirement



Requirement is
structured into user
stories +
acceptance criteria



The same
structured context
is given to AI



AI supports implementation
with the right goals and
permissions



Developer, QA, and customer
validation create better
future historical context



More accurate AI outputs, shared understanding, and repeatable delivery



Core Insights: The challenge was not legacy code. It was connecting the existing process and historical knowledge to AI in a structured way.

